

Vector konteinerio dokumentacija

Generated by Doxygen 1.16.1

1 Class Index	1
1.1 Class List	1
2 File Index	3
2.1 File List	3
3 Class Documentation	5
3.1 my::vector< T > Class Template Reference	5
3.1.1 Detailed Description	7
3.1.2 Constructor & Destructor Documentation	7
3.1.2.1 vector() [1/4]	7
3.1.2.2 vector() [2/4]	7
3.1.2.3 vector() [3/4]	8
3.1.2.4 vector() [4/4]	8
3.1.3 Member Function Documentation	8
3.1.3.1 at()	8
3.1.3.2 data()	8
3.1.3.3 empty()	9
3.1.3.4 insert()	9
3.1.3.5 operator=() [1/2]	9
3.1.3.6 operator=() [2/2]	10
3.1.3.7 operator[]()	10
3.1.3.8 push_back()	10
3.1.3.9 reserve()	11
3.1.3.10 resize()	11
3.1.3.11 size()	11
4 File Documentation	13
4.1 vector.hpp	13
Index	19

Chapter 1

Class Index

1.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

my::vector< T >		5
---------------------------------------	-------	-------------------

Chapter 2

File Index

2.1 File List

Here is a list of all documented files with brief descriptions:

vector.hpp	13
--------------------------------------	----

Chapter 3

Class Documentation

3.1 my::vector< T > Class Template Reference

```
#include <vector.hpp>
```

Public Types

- using **size_type** = int
- using **value_type** = T
- using **iterator** = T*
- using **const_iterator** = const T*

Public Member Functions

- **vector** ()
default konstruktorius
- **vector** (int s)
- **vector** (std::initializer_list< T > lst)
- template<typename InputIt>
vector (InputIt first, InputIt last)
Range constructor: sukuria vektoriu kopijuojant intervala [first, last).
- **vector** (const vector< T > &v)
- **vector** & **operator=** (const **vector**< T > &v)
- **vector** (vector< T > &&v)
- **vector** & **operator=** (**vector**< T > &&v)
- ~**vector** ()
Destruktorius, atlaisvina dinamine atmintyje saugoma masyva.
- size_type **size** () const
- int **max_size** () const
Griztina maksimalu galima dydzio skaiciu (apytiksliai).
- T & **operator[]** (int n)
- const T & **operator[]** (int n) const
Nepatikrintas indeksu operatorius (const versija).
- T & **at** (int n)
- const T & **at** (int n) const
- T & **front** ()

Grazina pirma elemento reference.

- `const T & front () const`
- `T & back ()`

Grazina paskutinio elemento reference.

- `const T & back () const`
- `bool empty () const`
- `T get (int n) const`

Getteris (kopija) elemento.

- `void set (int n, T val)`

Setteris elemento.

- `void reserve (int newalloc)`
- `size_type capacity () const`

Grazina kapaciteta (kiek vietos uzrezervuota).

- `void push_back (const T &val)`
- `void push_back (T &&val)`

Prideda elementa i pabaiga (rvalue).

- `void resize (int newsize, T val=T())`
- `void pop_back ()`

Pasalina paskutini elementa.

- `iterator begin ()`

Grazina iteratoriu i pradzia.

- `const_iterator begin () const`
- `const_iterator cbegin () const noexcept`
- `std::reverse_iterator< iterator > rbegin ()`
- `std::reverse_iterator< const_iterator > crbegin () const`
- `iterator end ()`

Grazina iteratoriu i pabaiga.

- `const_iterator end () const`
- `const_iterator cend () const noexcept`
- `std::reverse_iterator< iterator > rend ()`
- `std::reverse_iterator< const_iterator > crend () const`
- `iterator insert (iterator p, const T &val)`
- `iterator erase (iterator p)`
- `iterator erase (iterator first, iterator last)`
- `void clear ()`

Istrina visus elementus (neatlaisvina atminties).

- `T * data ()`

Uzpildo vektoriaus pirma n elementu reiksme.

- `const T * data () const`
- `void shrink_to_fit ()`

Sumazina rezervuota vieta iki dydzio.

- `void swap (vector &other) noexcept`

Apkeicia su kitu vector'iumi.

- `template<typename... Args>`
`void emplace_back (Args &&... args)`

In-place konstravimas paskutinio elemento.

- `template<typename... Args>`
`iterator emplace (iterator pos, Args &&... args)`
- `void assign (size_type n, const T &val)`

Uzpildo vektoriu n elementu su val.

- `template<typename InputIt>`
`void assign (InputIt first, InputIt last)`

Uzpildo vektoriu intervale [first,last).

- `void assign (std::initializer_list< T > ilist)`

Uzpildo vektoriu is initializer list.

3.1.1 Detailed Description

```
template<typename T>  
class my::vector< T >
```

Paprastas vektoriaus konteineris, imituoja std::vector

Template Parameters

<i>T</i>	elementu tipas
----------	----------------

3.1.2 Constructor & Destructor Documentation

3.1.2.1 vector() [1/4]

```
template<typename T>  
my::vector< T >::vector (  
    int s) [inline], [explicit]
```

Konstruktuoja vektoriu su pradiniu dydziu

Parameters

<i>s</i>	pradinis elementu skaicius
----------	----------------------------

Exceptions

<i>std::length_error</i>	jei $s < 0$
--------------------------	-------------

3.1.2.2 vector() [2/4]

```
template<typename T>  
my::vector< T >::vector (  
    std::initializer_list< T > lst) [inline]
```

Konstrukcija is initializer_list

Parameters

<i>lst</i>	sarasas elementu
------------	------------------

3.1.2.3 vector() [3/4]

```
template<typename T>
my::vector< T >::vector (
    const vector< T > & v)
```

Copy constructor

Parameters

<i>v</i>	kitas vector
----------	--------------

3.1.2.4 vector() [4/4]

```
template<typename T>
my::vector< T >::vector (
    vector< T > && v)
```

Move constructor

Parameters

<i>v</i>	movable vector
----------	----------------

3.1.3 Member Function Documentation**3.1.3.1 at()**

```
template<typename T>
T & my::vector< T >::at (
    int n)
```

Tikrintas prieigos metodus, metamas out_of_range jei uz ribu

Parameters

<i>n</i>	indeksas
----------	----------

Returns

reference i elementa

3.1.3.2 data()

```
template<typename T>
T * my::vector< T >::data () [inline]
```

Uzpildo vektoriaus pirma n elementu reiksme.

Grazina rodykle i vidini duomenu masyva

3.1.3.3 empty()

```
template<typename T>
bool my::vector< T >::empty () const [inline]
```

Patikrina ar vektorius tuscias

Returns

true jei tuscias

3.1.3.4 insert()

```
template<typename T>
vector< T >::iterator my::vector< T >::insert (
    iterator p,
    const T & val)
```

Iteratoriaus i pozicija p istrynimas

Parameters

p	iteratorius i istrinama elementa
-----	----------------------------------

Returns

iteratorius i elementa po istrynimo

3.1.3.5 operator=() [1/2]

```
template<typename T>
vector< T > & my::vector< T >::operator= (
    const vector< T > & v)
```

Copy assignment operator

Parameters

v	kitas vector
-----	--------------

Returns

reference i this

3.1.3.6 operator=() [2/2]

```
template<typename T>
vector< T > & my::vector< T >::operator= (
    vector< T > && v)
```

Move assignment

Parameters

<i>v</i>	movable vector
----------	----------------

Returns

reference i this

3.1.3.7 operator[]()

```
template<typename T>
T & my::vector< T >::operator[] (
    int n) [inline]
```

Nepatikrintas indeksio operatorius

Parameters

<i>n</i>	indekso pozicija
----------	------------------

Returns

reference i elementa

3.1.3.8 push_back()

```
template<typename T>
void my::vector< T >::push_back (
    const T & val)
```

Prideda elementa i pabaiga (lvalue)

Parameters

<i>val</i>	elementas
------------	-----------

3.1.3.9 reserve()

```
template<typename T>
void my::vector< T >::reserve (
    int newalloc)
```

Rezervuoja atminties vieta be elemento inicializacijos

Parameters

<i>newalloc</i>	naujas rezervuotas elementu kiekis
-----------------	------------------------------------

3.1.3.10 resize()

```
template<typename T>
void my::vector< T >::resize (
    int newsize,
    T val = T())
```

Pakeicia vektoriaus dydžio reikšmę, jei reikia inicializuoja naujus elementus

Parameters

<i>newsize</i>	naujas dydis
<i>val</i>	reikšmė naujiems elementams

3.1.3.11 size()

```
template<typename T>
size_type my::vector< T >::size () const [inline]
```

Grazina vektoriaus dydži (kiek elementu yra)

Returns

elementu skaičius

The documentation for this class was generated from the following file:

- vector.hpp

Chapter 4

File Documentation

4.1 vector.hpp

```
00001 #pragma once
00002
00003 #include <iostream>
00004 #include <stdexcept>
00005 #include <algorithm>
00006 #include <initializer_list>
00007 #include <iterator>
00008 #include <utility>
00009
00010 /*
00011 Atkuriamas STL std::vector tam kad ismokyti kurti savo konteineri
00012
00013 TODO:
00014
00015 Documentation
00016 */
00017 namespace my{
00018 template<typename T>
00022 class vector {
00023 private:
00024     int size_; //kiek elementu vektoriuje
00025     int space_; //kiek atminties uzrezervuota
00026     T* element_; //patys tie elementai (tiksliau pointeris i array prazia)
00027
00028 public:
00029     using size_type = int;
00030     using value_type = T;
00031     using iterator = T*;
00032     using const_iterator = const T*;
00033     vector() :size_{0} ,space_{0}, element_{nullptr} {}
00034
00035     explicit vector(int s) //constructor
00036         : size_{s}, space_{s}, element_{ new T[s] }
00037     {
00038         if(s < 0)
00039             throw std::length_error("size cannot be < 0 for a vector");
00040         for(int i=0; i<s; ++i)
00041             element_[i] = T{};
00042     }
00043
00044     vector(std::initializer_list<T> lst) // initializer list {} constructor
00045         : size_{static_cast<int>(lst.size())},
00046           space_{static_cast<int>(lst.size())},
00047           element_{new T[size_]}
00048     { std::copy(lst.begin(), lst.end(), element_); }
00049
00050     template<typename InputIt> vector(InputIt first, InputIt last)
00051         : size_{ static_cast<int>(std::distance(first, last)) },
00052           space_{ static_cast<int>(std::distance(first, last)) },
00053           element_{ new T[size_] }
00054     { std::copy(first, last, element_); } //range constructor
00055
00056     vector(const vector<T>& v); //copy constructor
00057
00058     vector& operator=(const vector<T>&v); //copy assignment
00059
00060
00061
00062
00063
00064
00065
00066
00067
00068
00069
00070
00071
```

```

00074     vector(vector<T>&& v);                                //move constructor
00075
00079     vector& operator=(vector<T>&& v);                      //move assignment
00080
00082     ~vector() { delete[] element_; }                     //destructor
00083
00086     size_type size() const { return size_; }              //current size of vector
00087
00089     int max_size() const { return INT_MAX / sizeof(T); }  //max size
00090
00094     T& operator[] (int n) { return element_[n]; }         //operatorius []
00096     const T& operator[] (int n) const { return element_[n]; }; //const versija ant const vektoriu
00097
00101     T& at(int n);                                         //checked access
00102     const T& at(int n) const;                             //const checked access
00103
00105     T& front() { return element_[0]; }                    //return first element reference
00106     const T& front() const { return element_[0]; }        //const version
00107
00109     T& back() { return element_[size_ - 1]; }             //return last element reference
00110     const T& back() const { return element_[size_ - 1]; } //const version
00111
00114     bool empty() const { return begin() == end(); }       //check if container empty
00115
00117     T get(int n) const { return element_[n]; }            //getteris
00119     void set(int n, T val) { element_[n] = val; }         //setteris
00120
00123     void reserve(int newalloc);                           //reservuoti naujos vietas
00124
00126     size_type capacity() const { return space_; }         //kiek vietas yra funk
00127
00130     void push_back(const T& val);                          //idet elementa T i gala vektorius
00132     void push_back(T&& val);
00133
00137     void resize(int newsize, T val = T());                //pakeist didy vektoriaus
00138
00140     void pop_back();                                       //remove last element of container
00141
00143     iterator begin() { return element_; }                  //pradziuos iteratorius
00144     const_iterator begin() const { return element_; }
00145     const_iterator cbegin() const noexcept { return element_; } //const pradziuos iteratorius
00146
00147     std::reverse_iterator<iterator> rbegin() { return std::reverse_iterator<iterator>(end()); }
00148     std::reverse_iterator<const_iterator> crbegin() const { return
std::reverse_iterator<const_iterator>(cend()); }
00149
00151     iterator end() { return element_ + size_; }            //galo iteratorius
00152     const_iterator end() const { return element_ + size_; }
00153     const_iterator cend() const noexcept { return element_ + size_; } //const galo iteratorius
00154
00155     std::reverse_iterator<iterator> rend() { return std::reverse_iterator<iterator>(begin()); }
//reverse iterator
00156     std::reverse_iterator<const_iterator> crend() const { return
std::reverse_iterator<const_iterator>(cbegin()); }
00157
00161     iterator insert(iterator p, const T& val);
00162     iterator erase(iterator p);
00163     iterator erase( iterator first, iterator last);
00164
00166     void clear() { size_ = 0; }                             //allocated space still belongs to the
container
00167
00169
00170
00172     T* data() { return element_; }                          //public access to element_
00173     const T* data() const { return element_; }             //const public access to element_
00174
00176     void shrink_to_fit();                                    //sumazint space_ = size_
00177
00179     void swap(vector& other) noexcept;
00180
00182     template<typename... Args>
00183     void emplace_back(Args&&... args);
00184
00185     template<typename... Args>
00186     iterator emplace(iterator pos, Args&&... args);
00187
00189     void assign(size_type n, const T& val);                 //fill assign
00191     template<typename InputIt>
00192     void assign(InputIt first, InputIt last);               //range assign
00194     void assign(std::initializer_list<T> ilist);           //initializer list assign
00195 };
00196
00197 template<typename T>
00198 bool operator==(const vector<T>& v1, const vector<T>& v2)
00199 {
00200     if (v1.size() != v2.size())

```

```

00201         return false;
00202     for(int i = 0; i < v1.size(); ++i)
00203         if(v1[i] != v2[i])
00204             return false;
00205     return true;
00206 }
00207
00208
00209
00210 template<typename T>
00211 bool operator!=(const vector<T>& v1, const vector<T>& v2)
00212 { return !(v1 == v2); }
00213
00214 template<typename T>
00215 bool operator<(const vector<T>& v1, const vector<T>& v2)
00216 {
00217     return std::lexicographical_compare(v1.begin(), v1.end(), v2.begin(), v2.end());
00218 }
00219
00220 template<typename T>
00221 bool operator<=(const vector<T>& v1, const vector<T>& v2)
00222 { return !(v2 < v1); }
00223
00224 template<typename T>
00225 bool operator>(const vector<T>& v1, const vector<T>& v2)
00226 { return v2 < v1; }
00227
00228 template<typename T>
00229 bool operator>=(const vector<T>& v1, const vector<T>& v2)
00230 { return !(v1 < v2); }
00231
00232 //copy constructor
00233 template<typename T>
00234 vector<T>::vector(const vector<T>& v)
00235     : size_(v.size_), space_(v.space_), element_(new T[v.size_])
00236     //allocatint memory praeito vectoriaus dydzio ir inicializuoti kopijuojant
00237     {
00238         std::copy(v.element_, v.element_ + v.size_, element_);
00239     }
00240
00241 //copy assignment
00242 template<typename T>
00243 vector<T>& vector<T>::operator=(const vector<T>& v)
00244 {
00245     //std::cerr << "copy assignment called\n";
00246     if (this==&v) return *this; //self assignment
00247
00248     if(v.size_<= space_) //jei nereik allocatint
00249     {
00250         for(int i = 0; i < v.size_; i++)
00251             element_[i] = v.element_[i];
00252         size_ = v.size_;
00253         return *this;
00254     }
00255
00256     T* p = new T[v.size_];
00257     for(int i = 0; i<v.size_; i++)
00258         p[i] = v.element_[i];
00259
00260     delete[] element_;
00261     space_ = size_ = v.size_;
00262     element_ = p;
00263     return *this;
00264 }
00265 //move constructor
00266 template <typename T>
00267 vector<T>::vector(vector<T>&& v)
00268     :size_{v.size_}, space_{v.space_}, element_{v.element_}
00269 {
00270     v.space_ = 0;
00271     v.size_ = 0;
00272     v.element_ = nullptr;
00273 }
00274 //move assignment
00275 template <typename T>
00276 vector<T>& vector<T>::operator=(vector<T>&& v)
00277 {
00278     if(this != &v){ //protection against self assignment
00279         delete[] element_;
00280         element_ = v.element_;
00281         size_ = v.size_;
00282         space_ = v.space_;
00283         v.element_ = nullptr;
00284         v.space_ = 0;
00285         v.size_ = 0;
00286     }
00287     return *this;

```

```

00288 }
00289
00290 template <typename T>
00291 void vector<T>::reserve(int newalloc)
00292 {
00293
00294     if (newalloc<=space_)           // never decrease allocation
00295         return;
00296     //std::cerr << "reserve called, newalloc=" << newalloc << " size_" << size_ << "\n";
00297     T* p = new T[newalloc];         // allocate new space
00298     for (int i=0; i<size_; ++i)     // copy old elements
00299         p[i] = element_[i];
00300     delete[] element_;              // deallocate old space
00301     element_ = p;
00302     space_ = newalloc;
00303 }
00304
00305 template<typename T>
00306 void vector<T>::resize(int newsize, T val)
00307 {
00308     reserve(newsize);
00309
00310     for(int i = size_; i < newsize; i++)
00311         element_[i] = val;
00312     size_ = newsize;
00313 }
00314
00315 template<typename T>
00316 void vector<T>::push_back(const T& val)
00317 {
00318     if(space_ == 0)
00319         reserve(8);
00320     else if(size_ == space_)
00321         reserve(2*space_);
00322     element_[size_] = val;
00323     size_++;
00324 }
00325
00326 template<typename T>
00327 void vector<T>::push_back(T&& val)
00328 {
00329     if(space_ == 0)
00330         reserve(8);
00331     else if(size_ == space_)
00332         reserve(2*space_);
00333     element_[size_] = std::move(val);
00334     size_++;
00335 }
00336
00337 template<typename T>
00338 void vector<T>::pop_back()
00339 {
00340     if(size_ == 0)
00341         throw std::out_of_range("used pop_back() on empty vector!");
00342     size_--;
00343 }
00344
00345 template<typename T>
00346 T& vector<T>::at(int n)
00347 {
00348     if (n<0 || size_<= n)
00349         throw std::out_of_range("vector::at accessed element out of range");
00350     return element_[n];
00351 }
00352
00353 template<typename T>
00354 const T& vector<T>::at(int n) const
00355 {
00356     if (n<0 || size_<= n)
00357         throw std::out_of_range("vector::at accessed element out of range");
00358     return element_[n];
00359 }
00360
00361 template<typename T>
00362 typename vector<T>::iterator vector<T>::erase(iterator p)
00363 {
00364     if(p == end())
00365         return p;
00366     std::move(p+1, end(), p); //move elements from p+1, until end() to p
00367     --size_;
00368     return p;
00369 }
00370
00371 template<typename T>
00372 typename vector<T>::iterator vector<T>::erase(iterator first, iterator last)
00373 {
00374     if(first == last)

```

```

00375         return first;
00376     auto n = last - first;
00377     std::move(last, end(), first);
00378     size_ -= n;
00379     return first;
00380 }
00381
00382 template<typename T>
00383 typename vector<T>::iterator vector<T>::insert(iterator p, const T& val)
00384 {
00385     int index = p - begin(); //jeigu reserve pakeistu vektoriaus vieta atmintyje
00386     if(size() == capacity())
00387         reserve(size() == 0?8:2*size());
00388     p = begin() + index; //uzdet p kur pries tai buvo relative to vector index
00389     std::move_backward(p, end(), end() + 1);
00390     *(begin() + index) = val;
00391     size_++;
00392     return p;
00393 }
00394
00395 template<typename T>
00396 void vector<T>::shrink_to_fit()
00397 {
00398     if(size_ == space_)
00399         return;
00400     T* p = new T[size_];
00401     std::move(begin(), end(), p);
00402     delete[] element_;
00403     element_ = p;
00404     space_ = size_;
00405 }
00406
00407 template<typename T>
00408 void vector<T>::swap(vector& other) noexcept
00409 {
00410     std::swap(element_, other.element_);
00411     std::swap(space_, other.space_);
00412     std::swap(size_, other.size_);
00413 }
00414
00415 template<typename T>
00416 void vector<T>::assign(size_type n, const T& val)
00417 {
00418     if(n > space_)
00419         reserve(n);
00420     for(int i = 0; i < n; i++)
00421         element_[i] = val;
00422     size_ = n;
00423 }
00424
00425 template<typename T>
00426 template<typename InputIt>
00427 void vector<T>::assign(InputIt first, InputIt last)
00428 {
00429     int n = static_cast<int>(std::distance(first, last));
00430     if(n > space_)
00431         reserve(n);
00432     std::copy(first, last, element_);
00433     size_ = n;
00434 }
00435
00436 template<typename T>
00437 void vector<T>::assign(std::initializer_list<T> ilist)
00438 {
00439     int n = ilist.size();
00440     if(n > space_)
00441         reserve(n);
00442     std::copy(ilist.begin(), ilist.end(), element_);
00443     size_ = n;
00444 }
00445
00446 template<typename T>
00447 template<typename... Args>
00448 void vector<T>::emplace_back(Args&&... args)
00449 {
00450     if(space_ == 0)
00451         reserve(8);
00452     else if(size_ == space_)
00453         reserve(2*space_);
00454     new(&element_[size_]) T(std::forward<Args>(args)... );
00455     ++size_;
00456 }
00457
00458 template<typename T>
00459 template<typename... Args>

```

```
00462 typename vector<T>::iterator vector<T>::emplace(iterator pos, Args&&... args)
00463 {
00464     int index = pos - begin();
00465     if(size() == capacity())
00466         reserve(size() == 0?8:2*size());
00467     pos = begin() + index; //uzdet p kur pries tai buvo relative to vector index
00468     std::move_backward(pos, end(), end() + 1);
00469     new(&*pos) T(std::forward<Args>(args)... );
00470     size_++;
00471     return pos;
00472 }
00473 }
```

Index

at

my::vector< T >, 8

data

my::vector< T >, 8

empty

my::vector< T >, 8

insert

my::vector< T >, 9

my::vector< T >, 5

at, 8

data, 8

empty, 8

insert, 9

operator=, 9

operator[], 10

push_back, 10

reserve, 10

resize, 11

size, 11

vector, 7, 8

operator=

my::vector< T >, 9

operator[]

my::vector< T >, 10

push_back

my::vector< T >, 10

reserve

my::vector< T >, 10

resize

my::vector< T >, 11

size

my::vector< T >, 11

vector

my::vector< T >, 7, 8